

Practical - 4: Clustering

Implement K-Means and hierarchical clustering on `sales_data_sample.csv` and determine a suitable number of clusters using the elbow method.

Step 1 : Importing Libraries

```
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
```

Step 2 : Load Dataset

```
df = pd.read_csv("sales_data_sample.csv", encoding="latin1",
on_bad_lines="skip")
print("Dataset shape:", df.shape)
print(df.head())
```

Step 3 : Select numerical features for clustering

```
numeric_features = df.select_dtypes(include="number").copy()
numeric_features = numeric_features.replace([float("inf"), float("-inf")],
pd.NA).dropna()
```

```
print("\nNumerical features used for clustering:")
print(numeric_features.columns.tolist())
print(numeric_features.head())
```

Dataset shape: (2823, 25)

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	\
0	10107	30	95.70	2	2871.00	
1	10121	34	81.35	5	2765.90	
2	10134	41	94.74	2	3884.34	
3	10145	45	83.26	6	3746.70	
4	10159	49	100.00	14	5205.27	

	ORDERDATE	STATUS	QTR_ID	MONTH_ID	YEAR_ID	...	\
0	2/24/2003 0:00	Shipped	1	2	2003	...	
1	5/7/2003 0:00	Shipped	2	5	2003	...	
2	7/1/2003 0:00	Shipped	3	7	2003	...	
3	8/25/2003 0:00	Shipped	3	8	2003	...	
4	10/10/2003 0:00	Shipped	4	10	2003	...	

	ADDRESSLINE1	ADDRESSLINE2	CITY	STATE	\
0	897 Long Airport Avenue	NaN	NYC	NY	

1	59 rue de l'Abbaye	NaN	Reims	NaN
2	27 rue du Colonel Pierre Avia	NaN	Paris	NaN
3	78934 Hillside Dr.	NaN	Pasadena	CA
4	7734 Strong St.	NaN	San Francisco	CA

	POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME	DEALSIZE
0	10022	USA	NaN	Yu	Kwai	Small
1	51100	France	EMEA	Henriot	Paul	Small
2	75508	France	EMEA	Da Cunha	Daniel	Medium
3	90003	USA	NaN	Young	Julie	Medium
4	NaN	USA	NaN	Brown	Julie	Medium

[5 rows x 25 columns]

Numerical features used for clustering:

['ORDERNUMBER', 'QUANTITYORDERED', 'PRICEEACH', 'ORDERLINENUMBER', 'SALES', 'QTR_ID', 'MONTH_ID', 'YEAR_ID', 'MSRP']

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	QTR_ID
0	10107	30	95.70	2	2871.00	1
1	10121	34	81.35	5	2765.90	2
2	10134	41	94.74	2	3884.34	3
3	10145	45	83.26	6	3746.70	3
4	10159	49	100.00	14	5205.27	4

	MONTH_ID	YEAR_ID	MSRP
0	2	2003	95
1	5	2003	95
2	7	2003	95
3	8	2003	95
4	10	2003	95

Step 4 : Standardize data

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(numeric_features)
print("Standardized data shape:", X_scaled.shape)
```

Standardized data shape: (2823, 9)

Step 5 : K-Means clustering

```
wcss = []
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)
```

```
plt.figure(figsize=(6, 4))
plt.plot(range(1, 11), wcss, marker="o")
```

```

plt.title("Elbow Method")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia (WCSS)")
plt.tight_layout()
plt.savefig("code_4_elbow_method.png")
plt.close()

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
clusters_kmeans = kmeans.fit_predict(X_scaled)
clustered_df = df.loc[numeric_features.index].copy()
clustered_df["KMeans_Cluster"] = clusters_kmeans

plt.figure(figsize=(7, 5))
for cluster_id in sorted(clustered_df["KMeans_Cluster"].unique()):
    cluster_rows = clustered_df[clustered_df["KMeans_Cluster"] == cluster_id]
    plt.scatter(cluster_rows["SALES"], cluster_rows["PRICEEACH"],
label=f"Cluster {cluster_id}", alpha=0.7)
plt.title("K-Means Clustering")
plt.xlabel("SALES")
plt.ylabel("PRICEEACH")
plt.legend()
plt.tight_layout()
plt.savefig("code_4_kmeans_clusters.png")
plt.close()

print("\nElbow WCSS values:", [round(value, 2) for value in wcss])
print("\nK-Means cluster centers:")
print(pd.DataFrame(kmeans.cluster_centers_,
columns=numeric_features.columns))
print("\nK-Means cluster counts:")
print(clustered_df["KMeans_Cluster"].value_counts().sort_index())
print("\nPlots saved: code_4_elbow_method.png, code_4_kmeans_clusters.png")

```

Elbow WCSS values: [25407.0, 20090.89, 16909.33, 14818.01, 13541.41, 12547.11, 11743.69, 11067.29, 10529.86, 10085.26]

K-Means cluster centers:

	ORDERNUMBER	QUANTITYORDERED	PRICEEACH	ORDERLINENUMBER	SALES	\
0	0.805753	0.154271	0.158539	-0.099048	0.137189	
1	-0.263803	0.039507	0.705798	0.015109	0.535064	
2	-0.308327	-0.176570	-1.050030	0.060515	-0.809798	

	QTR_ID	MONTH_ID	YEAR_ID	MSRP
0	-1.153125	-1.155904	1.181622	0.070180
1	0.509145	0.506268	-0.444983	0.624008
2	0.269399	0.275407	-0.376264	-0.871648

K-Means cluster counts:

KMeans_Cluster

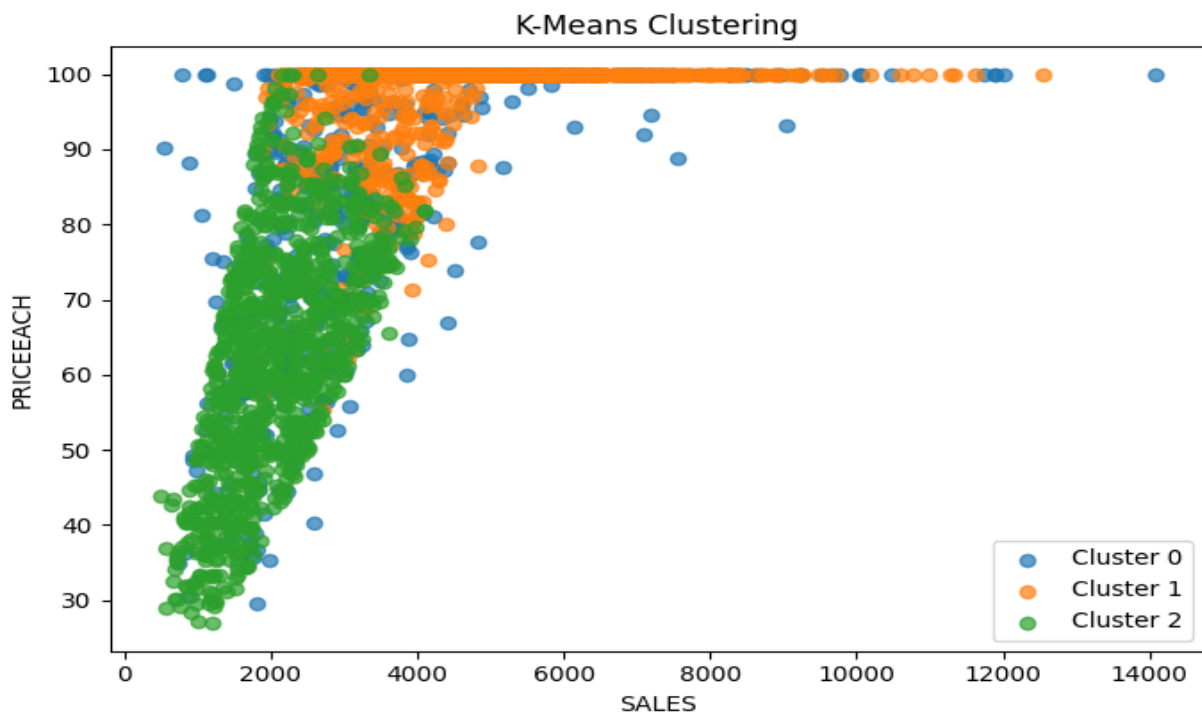
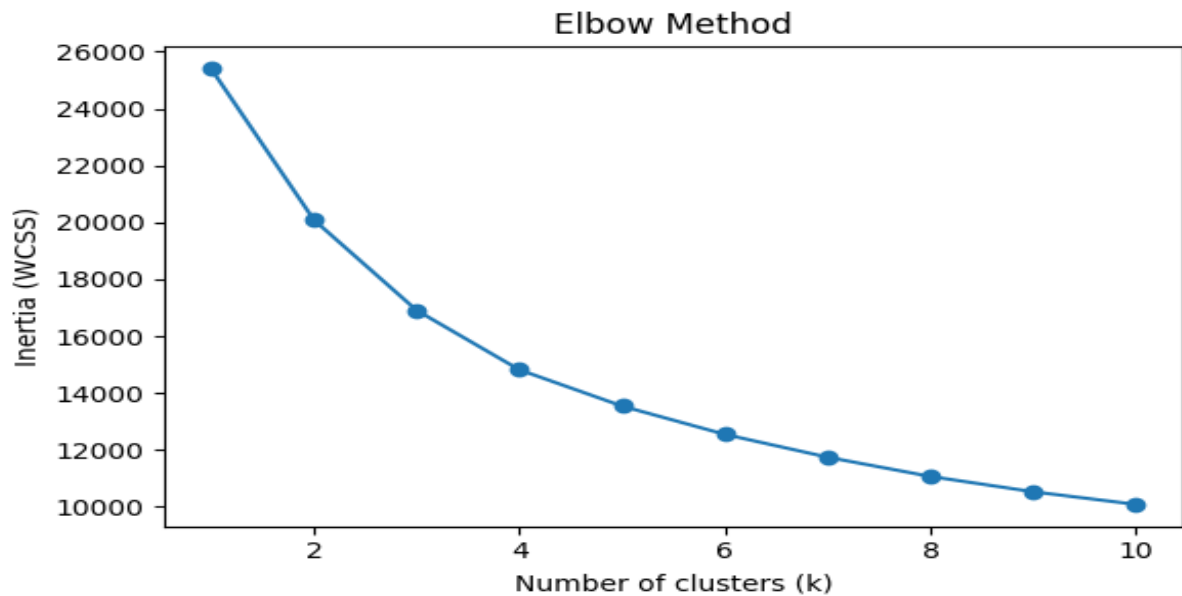
0 734

1 1183

2 906

Name: count, dtype: int64

Plots saved: code_4_elbow_method.png, code_4_kmeans_clusters.png



Step 6 : Hierarchical clustering

```
Z = linkage(X_scaled, method="ward")

plt.figure(figsize=(10, 5))
dendrogram(Z, no_labels=True)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.tight_layout()
plt.savefig("code_4_dendrogram.png")
plt.close()

clusters_hier = fcluster(Z, t=3, criterion="maxclust")
clustered_df["Hierarchical_Cluster"] = clusters_hier

print("\nHierarchical cluster counts:")
print(clustered_df["Hierarchical_Cluster"].value_counts().sort_index())
print("Plot saved: code_4_dendrogram.png")
```

Hierarchical cluster counts:

Hierarchical_Cluster

1 1655

2 478

3 690

Name: count, dtype: int64

Plot saved: code_4_dendrogram.png

